

Module 9 Week B — Core Skills Drill: Cypher

Warm-ups & SPARQL→Cypher Translation

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

This drill is a retrieval-under-pressure warm-up for Monday's lab. Part A gives you three short Cypher MATCH queries against a tiny recipe sub-graph so the M9B vocabulary (`:Recipe`, `:Cuisine`, `:Ingredient`, `{:OF_CUISINE}`, `{:USKS_INGREDIENT}`, `{:SUBCLASS_OF}`) becomes muscle memory. Part B is a **SPARQL→Cypher Translation Task** on the books mini-graph carried over from Week A — same five questions, same data, different query language. The autograder asserts that your Cypher returns the same result set as the W9A SPARQL original, so the equivalence between the two notations stops being a claim from the reading and becomes something you have proven with running code.

Setup

If you haven't already installed the Module 9 Week B packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

Windows users. Commands below assume Bash/Zsh (macOS/Linux/WSL2/Git Bash). On PowerShell or CMD, activate your venv with `.venv\Scripts\activate.ps1` (PowerShell) or `.venv\Scripts\activate.bat` (CMD), and confirm Docker Desktop is running with the WSL2 backend before `docker compose up -d`. Full cross-platform command tables are in the Pre-Work install plan.

Submission flow — all Module 9 Week B assignments. Week B uses a fork-and-submit flow (drill, lab, integration, both stretches). Steps 1–3 below replace the Classroom “Accept the assignment” step from earlier modules; the rest of the workflow is the same.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m9-d9b and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave “Copy the main branch only” checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab and click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m9-d9b.git
cd m9-d9b
```
- Create a branch:** `git checkout -b drill-9b-cypher-translation`.
- Start Neo4j locally** (from the repo root):

```
docker compose up -d
docker compose logs neo4j | grep Started
```

Wait until you see `Started` in the log. Default credentials are `neo4j / testtest` (set in `docker-compose.yml`).
- Install dependencies** (cached from the reading setup — should be fast):

```
pip install -r requirements.txt
```
- Load the fixtures:**

```
python load_fixture.py
```

A successful load prints `Fixture loaded: 23 nodes, 21 relationships ...`

What you'll do

Two parts, both autograded by `tests/test_drill_9b.py`:

- Part A — Cypher warm-ups** (`queries/warmups.py`). Three parameterized Cypher queries against a 5-recipe sub-graph (`data/recipes_mini.cypher`). Lightweight on its own; the point is to put the M9B vocabulary into your fingers before you touch the Translation Task.
- Part B — SPARQL→Cypher Translation Task** (`queries/translatiions.py`). Five Cypher queries (`q1()...q5()`) against a books mini-graph (`data/books_kg.cypher`) that mirrors the W9A drill's `fixtures/mini_kg.ttl`. The autograder runs the original W9A SPARQL queries against an rdflib parse of the books data, runs your Cypher against Neo4j, and asserts set-of-tuples equality on the named result columns. If your Cypher omits a row that the SPARQL returns, or returns an extra row it shouldn't, the test fails with a diff.

This is the Rosetta-stone gate. The Reading's [SPARQL ↔ Cypher mapping table](#) tells you the two notations express the same graph primitive. The Translation Task makes that claim falsifiable: build-time, against fixtures, with both engines running side-by-side. You don't just *read* that SPARQL and Cypher are equivalent — you *prove it* on every push.

Part A — Cypher warm-ups

Implement the three functions in `queries/warmups.py`. Each function's docstring names the expected result-set shape; the autograder compares your rows against gold on `recipes_mini.cypher`.

`q1_list_recipes() -> str`

Return a Cypher string that matches every `:Recipe` and returns its name. Result-set shape: one column named `name`, one row per recipe.

```
MATCH (r:Recipe)
RETURN r.name AS name
```

That's the whole query — the warm-up is making sure you produce the exact `AS name` column the autograder expects.

`q2_filter_by_cuisine(cuisine_name: str) -> tuple[str, dict]`

Return `(cypher_string, params_dict)`. Match recipes whose **direct** cuisine matches `$cuisine`. No hierarchy traversal in this query — that's Q3. Use `$cuisine` in the Cypher, never f-string interpolation.

The form to internalize:

```
MATCH (r:Recipe)-[:OF_CUISINE]->(c:Cuisine {name: $cuisine})
RETURN r.name AS name
```

...paired with `{ "cuisine": cuisine_name }`.

`q3_subclass_traversal(cuisine_name: str) -> tuple[str, dict]`

Same shape as Q2 (column `name`), but the cuisine match must also pick up sub-cuisines via `:SUBCLASS_OF`. On the fixture, Sichuan `{:SUBCLASS_OF}` Chinese, so `q3_subclass_traversal("Chinese")` should return both the Chinese recipes and the Sichuan recipes — while `q2_filter_by_cuisine("Chinese")` should return only the direct-Chinese ones.

Use `{:SUBCLASS_OF*0..}` so the cuisine itself and any ancestor can match in a single pattern:

```
MATCH (r:Recipe)-[:OF_CUISINE]->(:Cuisine)-[:SUBCLASS_OF*0..]->(:Cuisine {name: $
RETURN r.name AS name
```

`*0..` is the load-bearing detail — `*1..` would match only descendants, not the cuisine itself.

Part B — SPARQL → Cypher Translation Task

Implement `q1()...q5()` in `queries/translatiions.py`. Each function returns a Cypher string. The autograder runs the W9A SPARQL of the same number against an rdflib graph of the books data, runs your Cypher against Neo4j, and asserts the two result sets are equal as sets of tuples on the named columns. Row order is not asserted (except where a query specifies `ORDER BY` — none of Q1–Q5 do).

Each `q*` docstring names the exact column names the autograder expects — use `AS` to alias your returned values to those names. The books mini-graph has 6 authors, 5 books, and 2 topics; node ids look like `'book:1'`, `'author:3'`, `'topic:1'`.

One full worked translation — Q1. SPARQL form (from the W9A drill):

```
SELECT ?book ?title WHERE { ?book a :Book ; :title ?title . }
```

Cypher equivalent:

```
MATCH (b:Book)
RETURN b.id AS book, b.title AS title
```

Expected on the fixture: 5 rows. Note that `?book` in SPARQL is a URI/IRI, and the autograder compares against the book's `id` string (`'book:1'`, etc.) — so alias `b.id` AS `book`, not `b AS book`.

Q2 — filter on year. SPARQL `FILTER ( ?year > 2010 )` is **strict** greater-than. The Cypher equivalent is `WHERE b.year > 2010`. Columns: `book` (id string), `year` (int). On the fixture: 1 row.

Q3 — multi-row join. Traverse `(:Book)-[:AUTHORED_BY]->(:Author)`. Books with multiple authors produce multiple rows. Columns: `book` (id string), `author_name` (the Author's `name` property). On the fixture: 7 rows.

Q4 — OPTIONAL. SPARQL `OPTIONAL { ?book :topic ?topic }` translates to Cypher `OPTIONAL MATCH`. Every book appears in the result; books with no topic edge get `NULL` for `topic`. Columns: `book` (id string), `topic` (string or NULL). On the fixture: 5 rows; one row has `topic = NULL` (book:2). Pick one canonical source for the topic value — the `topic` property or the `:ON_TOPIC` edge — and document the choice in your PR description; the fixture aligns them. The autograder compares result rows as a set, so it would silently collapse duplicates if you returned both edges — but the modelling discipline is to pick one source so the query is unambiguous.

Q5 — ASK / existence. SPARQL `ASK { ?b :authored_by ?a1, ?a2 . FILTER ( ?a1 != ?a2 ) }` returns a single boolean. In Cypher, return a one-row, one-column boolean named `result`. Either `EXISTS { ... }` or a count-based predicate works — the autograder only checks the boolean value, not the predicate shape. On the fixture: `TRUE` (both `book:1` and `book:5` have two authors).

The Reading's [SPARQL ↔ Cypher mapping table](#) gives the row-by-row mapping for SELECT, WHERE, FILTER, OPTIONAL, and ASK. Keep it open while you write Q1–Q5.

Part C — Learner-written tests

Open `learner_tests/test_warmups.py`. Two test stubs are provided, each with a `pytest.fail("Not implemented...")` body. Replace each body with at least one real assertion against your warm-up queries.

The `driver` fixture (from `conf/test.py`) points at the same Neo4j instance the autograder uses, with the drill fixtures already loaded. The shape of a test:

```
def test_q1_list_recipes_returns_all_five(driver):
    cypher = q1_list_recipes()
    with driver.session() as session:
        rows = [record["name"] for record in session.run(cypher)]
        assert len(rows) == 5
        assert "Spaghetti Carbonara" in rows # or whichever recipe is in your fixture
```

The autograder's `test_learner_tests_complete` check parses your test file with `ast` and verifies (1) at least 2 test functions exist, (2) each contains at least one `assert`, (3) no bare `pass` bodies remain, and (4) no `pytest.fail("Not implemented")` placeholders remain. A test that runs Cypher but never asserts anything counts as a silent pass and will be rejected.

Run the tests

From the repo root, with Neo4j running and fixtures loaded:

```
pytest tests/ -v
```

The autograder's Translation-Task equivalence assertion is the gate to watch. For each `q1()...q5()`, the test:

- Loads `fixtures/mini_kg.ttl` (the W9A SPARQL fixture, vendored into this repo) into rdflib and runs the W9A SPARQL query → result set A.
- Connects to Neo4j (with `data/books_kg.cypher` already loaded) and runs your Cypher → result set B.
- Normalizes both to sets of tuples on the named columns and asserts `A == B`.

When the assertion fails, the test prints the symmetric difference — rows in SPARQL-not-in-Cypher and rows in Cypher-not-in-SPARQL — so you can see immediately which translation is off.

The same suite runs in CI on every push to a non-`main` branch (`.github/workflows/m9-d9b-autograder.yml`). The CI workflow brings up its own Neo4j service container; you do not need to do anything special for the autograder to run.

Common pitfalls

- Cypher injection via f-string interpolation.** Any time a value comes from outside the query (a function argument, anything the caller controls), use `$param` syntax — never `f"...{cuisine_name}..."`. The autograder rejects f-string Cypher in `queries/` via an AST check, and the habit itself is the same security defect as SPARQL injection.
- Forgetting to start Neo4j before running pytest locally.** `docker compose up -d` is part of every local run; the driver fixture will hang or fail with a connection error if Neo4j is not up.
- PR opened against upstream instead of your fork.** GitHub defaults the PR base to `LevelUp-Applied-AI/m9-d9b`; you must change it to your own fork. Actions are disabled on cross-fork PRs to upstream, so a misrouted PR will not run the autograder and cannot be merged.
- `q3_subclass_traversal` using `*1..` instead of `*0..`.** `*1..` requires at least one `:SUBCLASS_OF` hop, which excludes the cuisine itself — so `q3_subclass_traversal("Chinese")` would return Sichuan recipes but drop the direct-Chinese ones. Use `*0..`.
- Q4 returning duplicate rows when both `topic` property and `:ON_TOPIC` edge exist.** The autograder compares row sets, so duplicates collapse silently — but the modelling discipline is to pick one canonical source so the query is unambiguous and reproducible.
- Q5 returning the wrong column name.** The autograder expects a single column named `result`. Returning `RETURN true` (column will be named `true`) or `RETURN exists(...)` AS `answer` will fail equivalence on the column name.

Submit

- Push your branch:

```
git push -u origin drill-9b-cypher-translation
```
- Open a Pull Request *within your fork*** — base `main`, compare `drill-9b-cypher-translation`, **base repository = your fork** (GitHub defaults to upstream `LevelUp-Applied-AI/m9-d9b` — change it to your own fork).
- Wait for the autograder workflow on your PR to finish green.
- Your PR description must include:
 - One sentence on the canonical source you chose for Q4's `topic` value (the `topic` property or the `:ON_TOPIC` edge) and why.
 - Paste your PR URL into TalentLMS → Module 9 Week B → Core Skills Drill to submit this assignment.

